

Growmax Order Management Module

Technical Assessment Report

Stack: Laravel 12 · PHP 8.2+ · Sanctum · React 18 · Vite · Tailwind CSS · PHPUnit · Docker · k6

Contents

1	Introduction and Objective	2
1.1	The brief	2
1.2	Stack and constraints	2
2	Understanding the Requirements and Prioritization	2
2.1	Reading the brief as a dependency graph	2
2.2	Core vs. bonus	3
2.3	Planned vs. actual time allocation	3
2.4	Full project chronology	3
3	Design Phase	4
4	Architecture Overview	4
4.1	Backend: request flow	4
4.2	Where the state machine lives, and why	5
4.3	Moroccan phone: rule and normalisation	6
4.4	Frontend: request flow and auth	6
5	UML Modeling	7
6	Database Design	12
6.1	Tables and relationships	12
6.2	Indexes and the queries they serve	12
6.3	Seeding strategy	12
7	Backend Implementation Steps	12
7.1	Key decisions and trade-offs	13
8	Frontend Implementation Steps	13
8.1	Notable implementation details	14
9	Extra Touches Beyond the Brief	14
10	Testing and Quality Assurance	15
10.1	The automated test suite	15
10.2	Quality-assurance tooling	16
10.3	Load and concurrency testing (k6)	16
10.4	Reproducing the full QA pass	16
11	AI Usage and Transparency	17
12	Challenges and Debugging	17
12.1	Laravel 11 was uninstallable	17
12.2	Vite's default toolchain outpaced the requested stack	18
12.3	A monorepo restructure mid-build	18
12.4	An empty git add pathspec silently dropped a commit's contents	18

13 Deployment Journey and Infrastructure Decisions	18
13.1 Goal	18
13.2 Option 1: Laravel Cloud (abandoned for a non-technical reason)	18
13.3 Option 2: Render free tier via Docker (the final backend host)	19
13.4 Turning the free tier's constraint into a feature	19
13.5 Frontend hosting and the final architecture	19
13.6 Trade-offs accepted, consciously	20
14 Limitations and Possible Improvements	20
14.1 Honest gaps against the brief	20
14.2 Scaling to roughly 1,000,000 orders	20
14.3 Concurrent status updates	21
15 Installation Summary and Demo Credentials	21
16 Conclusion	22

1 Introduction and Objective

This report documents the design and implementation of the **Growmax Order Management Module**, a technical assessment project simulating the back-office order management system of a Moroccan e-commerce Cash-on-Delivery (COD) operation.

1.1 The brief

The assessment asked for a Laravel 11 REST API covering:

- Customer and order data models with a strict order status lifecycle (*New*, *Confirmed*, *Shipped*, *Delivered*, *Returned*, *Cancelled*);
- Moroccan mobile phone validation and normalisation;
- A paginated, filterable, sortable order listing endpoint with eager-loaded relations;
- A dashboard aggregate endpoint (counts per status, delivered COD total, delivery rate);
- Sanctum token authentication;
- Automated feature tests;

followed by a React frontend consuming that API, built against five reference screens (login, dashboard, orders list, create order, order details) supplied as a Stitch-generated static HTML/PNG design package.

1.2 Stack and constraints

The *core deliverable* – the full API and all five frontend screens – was built end-to-end in a single working session against a **seven-hour, one-day** time budget, split roughly evenly between the API and the frontend. Follow-up sessions then layered on strictly additive extras, a full quality-assurance and load-testing pass, and deployment (Sections 9, 10 and 13), none of which changed the core’s API contracts, state machine or validation rules. The final stack:

Layer	Technology
API framework	Laravel 12.64 (PHP 8.2+)
Auth	Laravel Sanctum 4.3 (bearer tokens)
Database	SQLite (dev default) / MySQL 8 (local MAMP)
Testing	PHPUnit, RefreshDatabase, in-memory SQLite
Frontend build	Vite 8, React 18.3
Routing	react-router-dom 6.30
HTTP client	axios 1.18
Styling	Tailwind CSS 3.4 (PostCSS)
Icons	lucide-react

One deliberate deviation from the brief is worth stating up front: Laravel 11 had reached end-of-life with an *unpatched* security advisory (CRLF injection, ID PKSA-mdq4-51ck-6kdq) covering its entire release range, which **composer** refuses to install without explicitly suppressing the advisory. I chose to target **Laravel 12** instead – structurally identical for every requirement in the brief, but with a clean **composer audit**. This is called out again in Section 12.

2 Understanding the Requirements and Prioritization

2.1 Reading the brief as a dependency graph

Before writing code, I separated the requirements into what the rest of the system depends on versus what depends on the rest of the system. Three pieces of business logic sit underneath everything else and were built first:

1. **The status state machine** – every other order operation (creation, listing, the dashboard, the frontend’s action buttons) needs to know what a valid order status is and what it may become next. Getting this wrong would ripple through every later layer.
2. **Moroccan phone normalisation** – customer identity in this domain *is* the phone number (used for order-customer matching and search), so the canonical format had to be settled before the order-creation endpoint could be written.
3. **The database schema and indexes** – the listing endpoint’s filters and sorts are only as good as the indexes backing them.

Everything else – CRUD endpoints, the dashboard aggregate, the React pages – was treated as a consumer of these three foundations rather than something to design independently.

2.2 Core vs. bonus

Core (built first, non-negotiable)	Bonus (built if time allowed)
State machine with DB-transaction-safe transitions	Debounced, cancellable search on the frontend
Moroccan phone rule + normalisation cast	Toast notifications, confirm dialogs
Server-side pagination/search/filter/sort with eager loading	Skeleton loading states, empty/error states on every view
Dashboard aggregate queries (no N+1, no full-table loads)	Unit tests for the phone rule and enum in isolation
Required feature tests (401, valid creation, invalid transition)	Mobile-responsive sidebar / off-canvas navigation

2.3 Planned vs. actual time allocation

The brief suggested working step by step (setup, database, auth, orders, state machine, phone rule, dashboard, tests, docs) and verifying with artisan commands after each step – I followed that sequence for the backend almost exactly, which is also the order the git history reflects (see Section 7). I allotted roughly:

Phase	Approx. share of the 7h
Backend (schema, auth, endpoints, state machine, tests, docs)	~45%
Frontend (setup, auth flow, pages, polish)	~45%
Design review + report/documentation	~10%

In practice the backend finished slightly under budget because the state machine and phone rule were validated with quick `tinker/curl` checks as I went rather than only at the end, which avoided a debugging pass later.

2.4 Full project chronology

The complete arc of the project, in the order the git history records it:

Phase	What happened
1. Requirements analysis	Brief decomposed into a dependency graph; core vs. bonus split (this section).
2. Design prototyping	Five reference screens generated (Stitch), reviewed, palette rebuilt (Section 3).
3. Backend	Scaffold → schema/enum/phone rule → API endpoints → tests → README (Section 7).
4. Frontend	API/auth foundation → shared components → shell/routing → five pages → README (Section 8).
5. First report + CORS fix	Initial version of this report; a real CORS bug (Vite auto-incrementing its dev port to 5174) root-caused and fixed.
6. Extra touches	Five additive features on top of the brief (Section 9).
7. Docker + deployment	Multi-stage <code>Dockerfile</code> , container verified locally, then the hosting journey of Section 13.
8. QA & load testing	Pint, Larastan level 6, three gap-filling tests, an N+1/index audit, three k6 scenarios, ESLint – full results in <code>TESTING_REPORT.md</code> and Section 10.

3 Design Phase

Before writing frontend code, I worked from an AI-generated design package (Stitch) covering five screens in a clean, Stripe/Linear-inspired SaaS direction, stored in `design/` alongside a `DESIGN.md` design system description:

1. **Login** – split screen, dark brand panel on the left, form on the right.
2. **Dashboard** – five KPI cards, an "Orders by Status" bar list, a recent-orders table.
3. **Orders list** – toolbar (search, status, city, sort), data table, pagination.
4. **Create order** – two-section form (customer information, order information).
5. **Order details** – order summary card, customer card, change-status actions, status-history timeline.

The reference HTML was treated strictly as a *layout and structure* guide, not as production code: it used the Tailwind CDN script and a Material-3-derived palette (bluish surfaces `#e5eeff`, a brownish outline `#8c7164`, a muddy primary `#9d4300`) that did not fit a "calm control" SaaS aesthetic. I refined the palette down to:

- background `#F8FAFC` (slate-50), white cards, slate-200 borders, 12px radius, ambient shadow;
- a *single* accent colour, orange `#F97316` (hover `#EA580C`), reserved for primary buttons, the active nav item, focus rings, links and timeline dots;
- a dark slate (`#0F172A`) sidebar with a white logo and an orange left-border active state;
- one consistent status-pill colour mapping used on every screen (blue / indigo / amber / green / red / slate for the six statuses).

This palette and the supporting shadow/radius scale were declared exactly once, as Tailwind theme extensions in `frontend/tailwind.config.js`, so no page repeats colour literals.

4 Architecture Overview

4.1 Backend: request flow

Every backend request follows the same path, keeping controllers thin and business logic centralised:

Layer	Responsibility
Route (<code>routes/api.php</code>)	Maps a verb + URI to a controller method, behind <code>auth:sanctum</code> except <code>/login</code> .
FormRequest	Validates and shapes input (<code>StoreOrderRequest</code> , <code>UpdateOrderStatusRequest</code> , <code>IndexOrderRequest</code> , <code>LoginRequest</code>).
Controller	Calls exactly one service method and wraps the result in an API Resource. No business logic.
Service	<code>OrderService</code> , <code>OrderStatusService</code> , <code>DashboardService</code> – own transactions, queries and side effects.
Enum	<code>OrderStatus</code> – owns the transition rules, independent of the database.
Model / Resource	Eloquent relationships and casts; API Resources shape the JSON response.

4.2 Where the state machine lives, and why

The transition rules live in `App\Enums\OrderStatus::allowedTransitions()` – a pure function with no database dependency:

```

1 public function allowedTransitions(): array
2 {
3     return match ($this) {
4         self::New => [self::Confirmed, self::Cancelled],
5         self::Confirmed => [self::Shipped, self::Cancelled],
6         self::Shipped => [self::Delivered, self::Returned],
7         self::Delivered, self::Returned, self::Cancelled => [],
8     };
9 }

```

Listing 1: `app/Enums/OrderStatus.php` – the state machine

I chose the enum over a controller `if/else` chain or a database-driven rules table for three reasons: (1) it is trivially unit-testable without a database, as shown by `OrderStatusTest`; (2) any code path that ever changes a status – HTTP request, artisan command, queued job, the seeder – is validated against the exact same rules, so there is no risk of the API and a future CLI tool disagreeing; (3) the frontend never has to hardcode the graph either, because the API exposes `allowed_next_statuses` computed from this same method on every order response.

The *side effects* of applying a transition – updating the order row and writing an audit record atomically – are a separate concern, handled by `OrderStatusService::transition()`:

```

1 return DB::transaction(function () use ($order, $target, $user) {
2     $locked = Order::whereKey($order->getKey())->lockForUpdate()->firstOrFail();
3     $current = $locked->status;
4
5     if (! $current->canTransitionTo($target)) {
6         throw InvalidStatusTransition::make($current, $target);
7     }
8
9     $locked->status = $target;
10    $locked->save();
11
12    return $locked->statusHistories()->create([
13        'previous_status' => $current,
14        'new_status' => $target,
15        'user_id' => $user->id,
16        'created_at' => now(),
17    ]);

```

```
18 });
```

Listing 2: `app/Services/OrderStatusService.php` – transactional transition

The row is re-read with `lockForUpdate()` *inside* the transaction rather than trusting the model instance passed in, so two concurrent requests racing to change the same order cannot both validate against a stale status and both succeed (see the concurrency discussion in Section 14). An illegal transition raises `InvalidStatusTransition`, a subclass of Laravel's `ValidationException` that also carries `allowed_next_statuses` in its JSON body, so a single exception type produces a normal 422 envelope the frontend already knows how to parse.

4.3 Moroccan phone: rule and normalisation

Phone handling is split into a static, dependency-free normaliser (`App\Support\MoroccanPhone`) and two thin wrappers around it: a Laravel validation Rule for FormRequests, and an Eloquent cast so the canonical form is enforced at the model layer no matter which code path writes a customer (API, seeder, `tinker`).

```
1 public static function normalize(?string $input): ?string
2 {
3     if ($input === null) return null;
4
5     $digits = preg_replace('/[^\d+]/', '', $input) ?? '';
6     if (str_starts_with($digits, '00')) {
7         $digits = '+' . substr($digits, 2);
8     }
9
10    $national = match (true) {
11        (bool) preg_match('/^\+212([67]\d{8})$/ ', $digits, $m) => $m[1],
12        (bool) preg_match('/^212([67]\d{8})$/ ', $digits, $m) => $m[1],
13        (bool) preg_match('/^0([67]\d{8})$/ ', $digits, $m) => $m[1],
14        default => null,
15    };
16
17    return $national === null ? null : '+212' . $national;
18 }
```

Listing 3: `app/Support/MoroccanPhone.php` – normalisation core

Only the 06/07 mobile prefixes are accepted – landlines (05) are rejected – because in a COD workflow the courier must be able to reach the customer by phone; a landline number does not serve that purpose. Separators (spaces, dashes, dots) are stripped before matching so "06 12-34 56 78" and 0612345678 normalise identically, and both 00212... and +212... international prefixes are accepted. The same helper backs order search, so a search term typed in either local or international format matches the stored +212... value.

4.4 Frontend: request flow and auth

The React SPA mirrors the same "thin surface, one source of truth" principle: `AuthContext` exposes `{ user, loading, login, logout }`, and a single axios instance in `src/lib/api.js` attaches the bearer token and reacts to 401s:

```
1 api.interceptors.request.use((config) => {
2     const token = tokenStore.get()
3     if (token) config.headers.Authorization = `Bearer ${token}`
4     return config
5 })
6
7 api.interceptors.response.use(
8     (response) => response,
```

```

9  (error) => {
10     const status = error.response?.status
11     if (status === 401 && !error.config?.url?.includes('/login')) {
12         tokenStore.clear()
13         if (window.location.pathname !== '/login') window.location.assign('/login')
14     }
15     return Promise.reject(error)
16 },
17 )

```

Listing 4: `frontend/src/lib/api.js` – token attachment and 401 handling

`ProtectedRoute` wraps every authenticated page; on app boot, if a token exists in `localStorage`, `AuthContext` calls `GET /me` to restore the session before rendering anything, showing a full-page loader in the interim rather than flashing the login screen.

5 UML Modeling

The following diagrams were generated with PlantUML directly from the implemented code – every actor, class, method and message below corresponds to a real route in `routes/api.php`, a real class in `app/`, or a real call in the React pages, not to a generic e-commerce template.

Figure 1 maps one use case to each real endpoint. The «include» relationships make a design decision visible: phone validation and status-history recording are not optional steps a caller may skip – creating an order *always* passes through the `MoroccanPhone` rule and *always* writes the opening history row, and changing a status *always* passes through the enum’s transition rules. The customer-lookup extra reuses the same phone normaliser as order creation rather than duplicating it.

Figure 2 shows the strict one-way dependency flow: controllers depend on `FormRequests` and services, services depend on models, and nothing points back up. The multiplicities encode the audit guarantee – every `Order` has *at least one* `OrderStatusHistory` row (the creation entry), never zero. The `OrderStatus` enum sits at the bottom with the transition API (`allowedTransitions()`, `canTransitionTo()`); both the service layer and the `UpdateOrderStatusRequest` depend on it, so there is no path from HTTP to the database that bypasses the state machine.

Figure 3 makes the concurrency design visible: the order row is re-read *inside* the transaction with `lockForUpdate()` before the state machine is consulted, so two racing requests serialise on the row lock and the loser re-validates against the winner’s committed status – the exact behaviour the k6 contention test of Section 10 verified empirically. The 422 for an illegal transition carries `allowed_next_statuses`, and the React page re-fetches rather than guessing, so the UI’s action buttons always reflect the server’s rules.

Figure 4 shows why an order can never exist without its customer or its opening audit row: customer upsert, order insert and the initial `OrderStatusHistory` entry all commit or roll back together in one `DB::transaction()`. Phone normalisation appears twice by design – once in validation (reject bad input early with a field-level 422) and once in the service (canonicalise before the customer match), both delegating to the same static `MoroccanPhone::normalize()`.

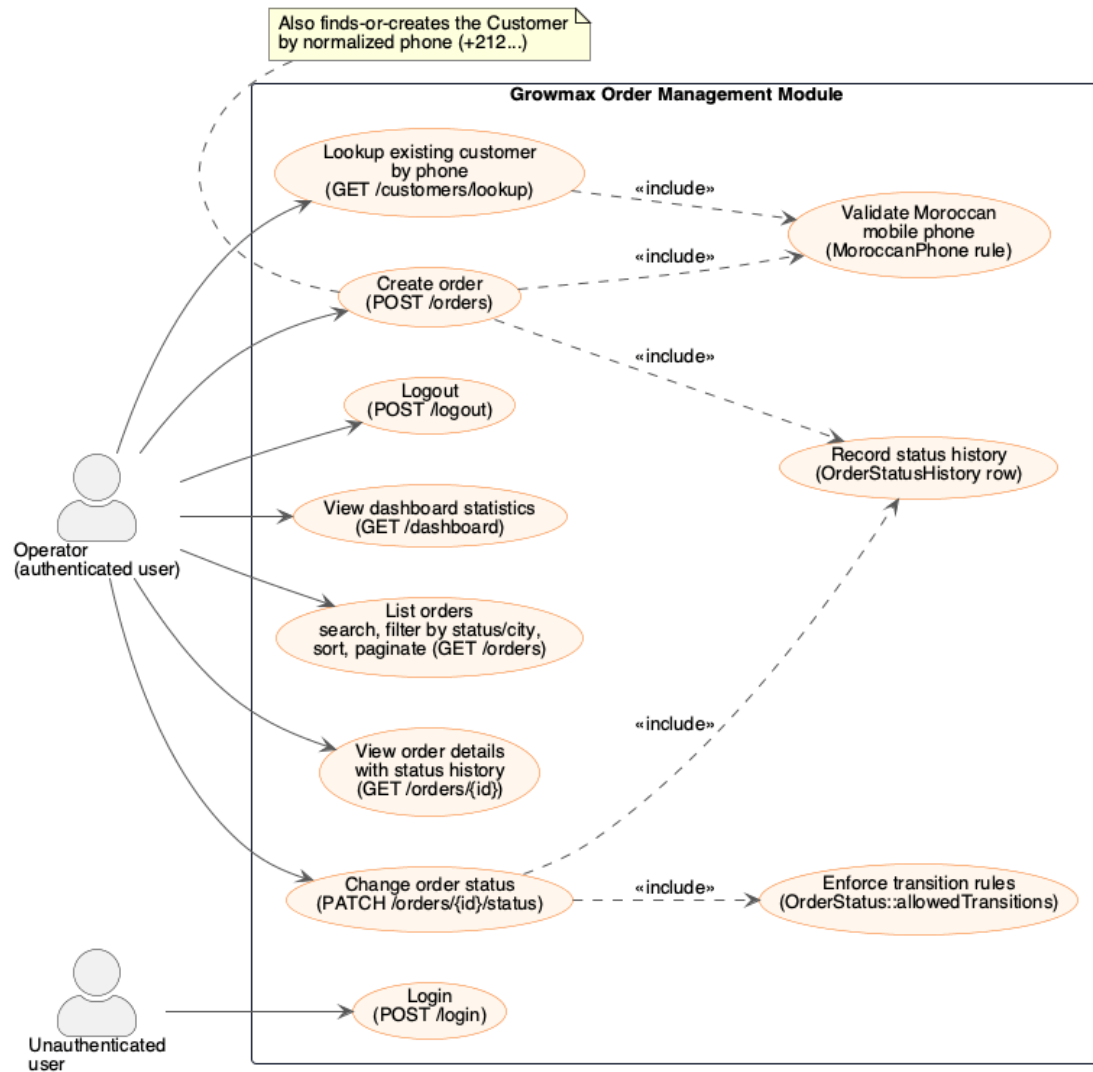


Figure 1: Use case diagram. The Operator (the authenticated demo user) can exercise every API capability; an unauthenticated user can only log in.

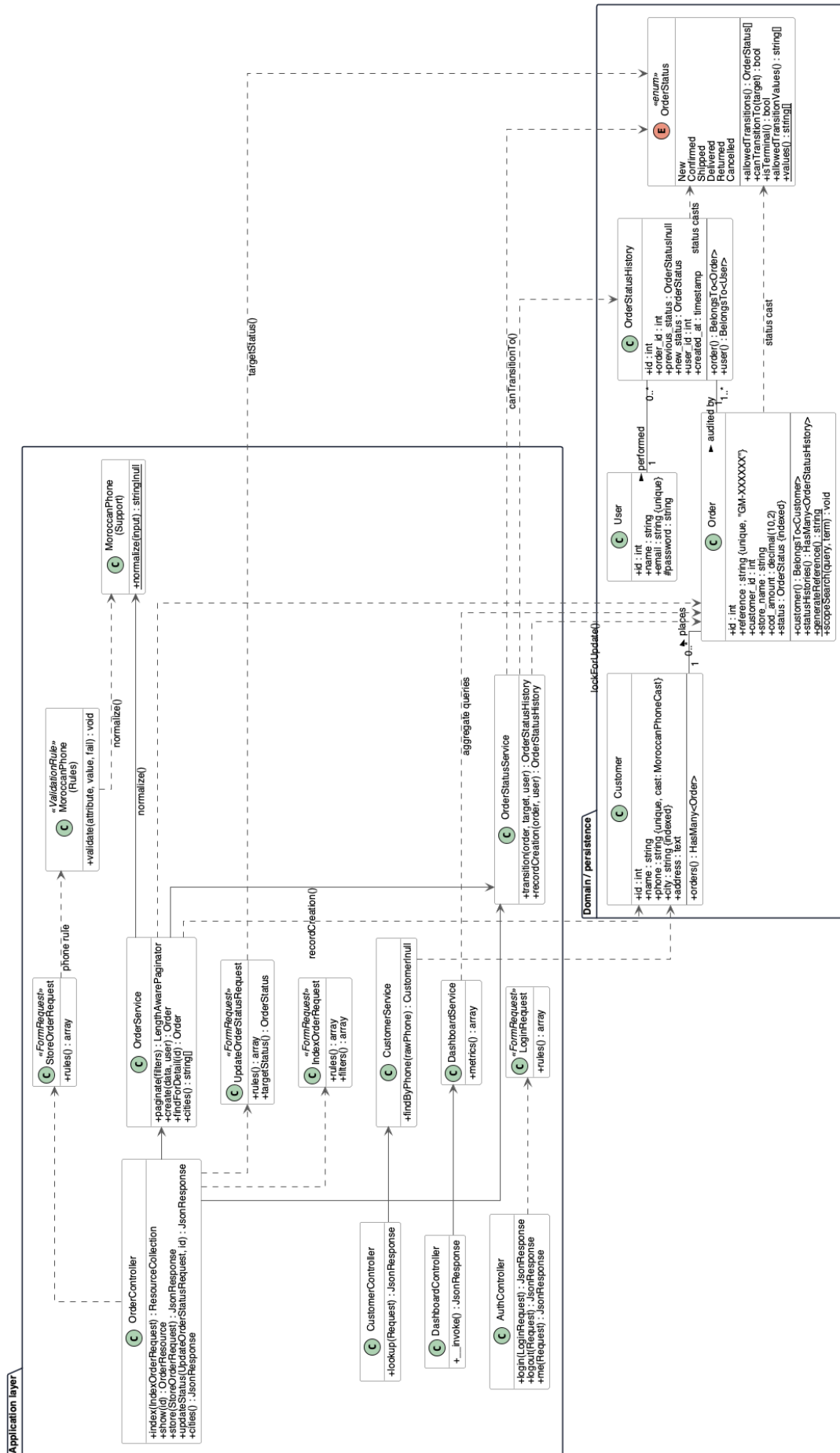


Figure 2: Class diagram: application layer (controllers, FormRequests, services, the phone rule) above, domain/persistence layer (Eloquent models and the OrderStatus enum) below, with attributes taken from the actual migrations.

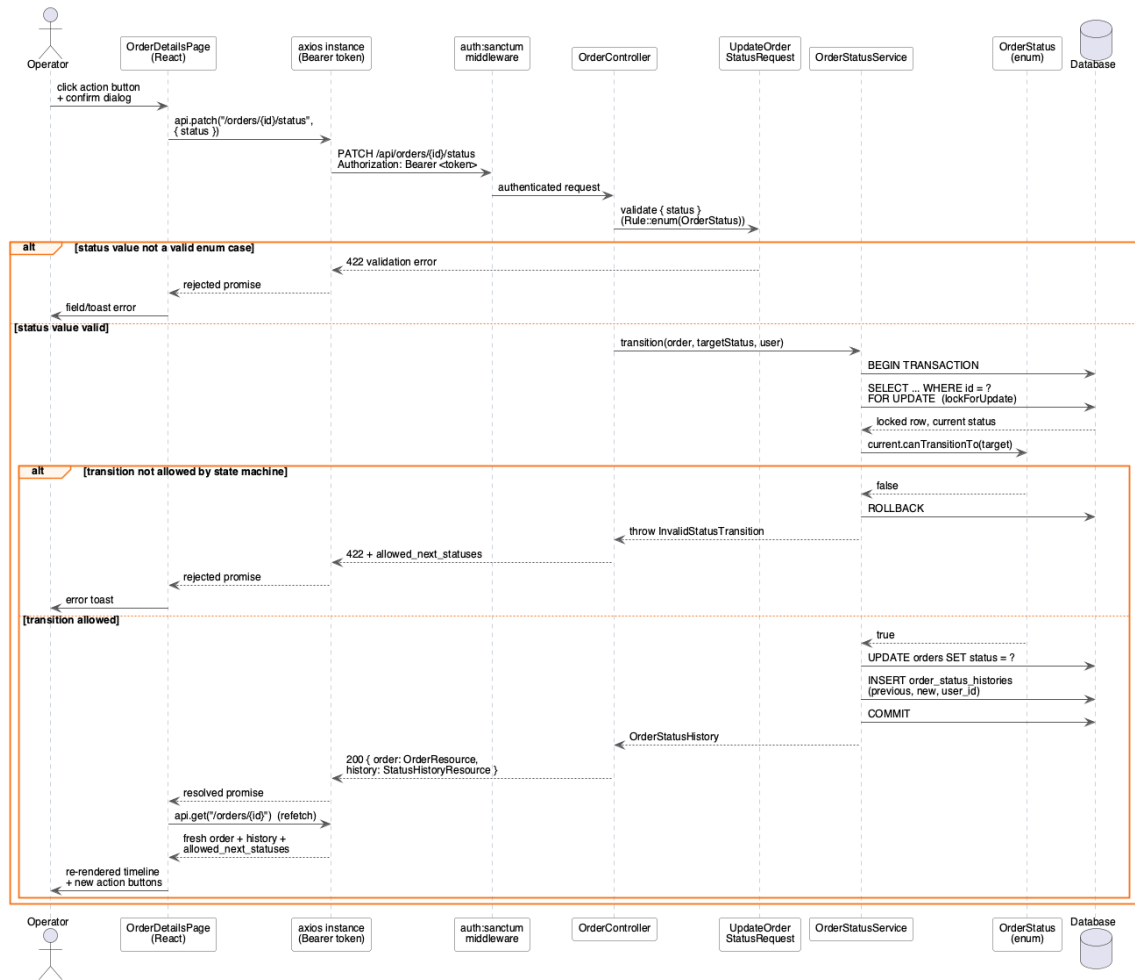


Figure 3: Sequence diagram: change order status – the flagship flow. Both rejection branches (invalid enum value at validation; legal-value-but-illegal transition inside the locked transaction) return a 422.

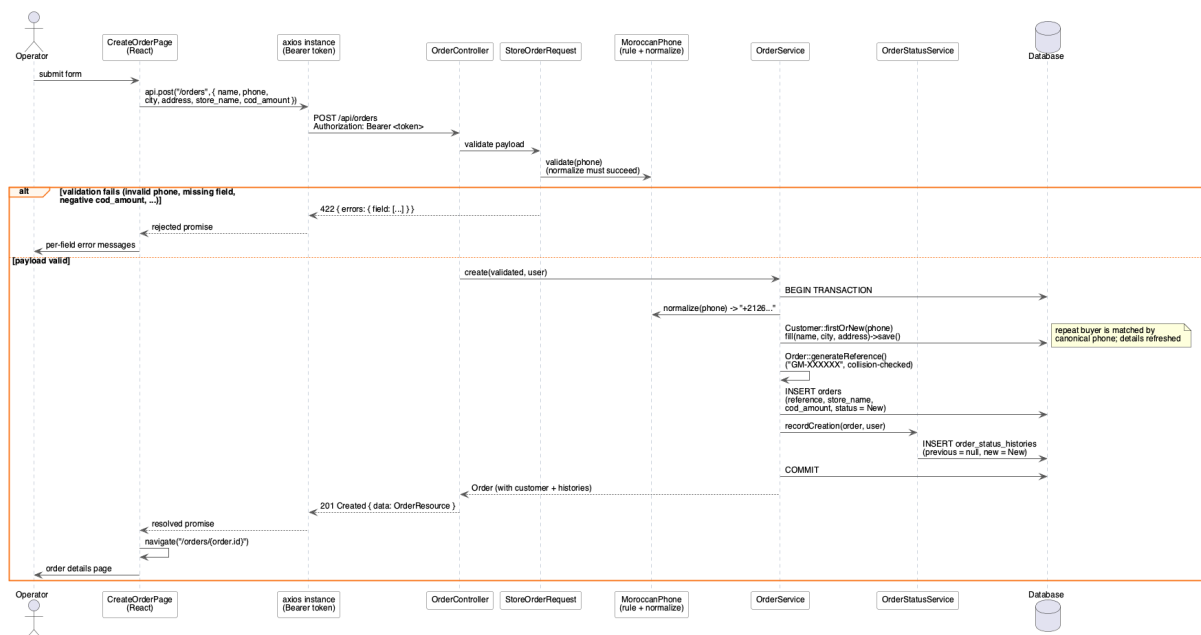


Figure 4: Sequence diagram: create order. Validation (including the `MoroccanPhone` rule) happens before the transaction; inside it, the customer is found-or-created by normalised phone and the opening history row is written atomically with the order.

6 Database Design

6.1 Tables and relationships

Table	Key columns	Relationships
customers	name, phone (unique), city, address	hasMany orders
orders	reference (unique), customer_id, store_name, cod_amount, status	belongsTo customer, hasMany statusHistories
order_status_histories	order_id, previous_status (nullable), new_status, user_id, created_at	belongsTo order, belongsTo user

6.2 Indexes and the queries they serve

Index	Query it supports
customers.phone (unique)	Customer-by-phone lookup on order creation (reuse vs. create), and the unique constraint doubles as an index for phone search.
customers.city	City filter on the orders list.
orders.reference (unique)	Reference search/lookup.
orders.customer_id	The belongsTo /eager-load join; added explicitly because SQLite, unlike MySQL, does not auto-index foreign keys.
orders.status	Status filter on the list and the dashboard's GROUP BY status .
orders.created_at	Default sort and the "last 3 months" seed spread.
orders.cod_amount	COD-amount sort.
orders(status, created_at) (composite)	The common case of filtering by status <i>and</i> sorting by date in one pass.
order_status_histories.order_id	Loading one order's full history, newest first.

6.3 Seeding strategy

The seeder creates one demo user, 35 customers with realistic Moroccan names and cities, and 115 orders distributed across all six statuses (weighted towards a healthy funnel: 22 New, 20 Confirmed, 18 Shipped, 35 Delivered, 9 Returned, 11 Cancelled). The interesting part is how each order's **order_status_histories** rows are produced: rather than hand-writing a plausible-looking chain per status, **DatabaseSeeder::pathToStatus()** runs a breadth-first search over **OrderStatus::allowedTransitions()** to compute the actual shortest legal path from **New** to the target status, then writes one history row per step with timestamps spread between the order's creation and now. This guarantees seeded history can never contradict the transition rules, even if the rules change later – there is no hand-maintained duplicate of the state machine in the seeder.

7 Backend Implementation Steps

The backend was built in the following order, matching the git history:

1. chore: scaffold Laravel 12 API project with SQLite – project bootstrap, confirmed a clean composer audit (see Section 12 for why Laravel 12 was chosen over 11).
2. feat(db): add customers, orders and status history schema – migrations, models, the **OrderStatus** enum, the Moroccan phone rule/cast, factories, and the state-machine-driven seeder, all added together since the seeder depends on the enum being final.

3. **feat(api): auth, orders, state machine, phone rule and dashboard** – Sanctum login/logout/me, the order CRUD + status endpoints, CORS configuration for the frontend's dev origin, and a `ForceJsonResponse` middleware so every error response (401/404/422/500) is guaranteed JSON rather than an HTML page, even when a client omits an `Accept` header.
4. **test: feature and unit coverage...** – the full test suite (Section 10).
5. **docs: backend README...** – install steps, endpoint table, and the "where do the rules live" explanation reused in this report.

7.1 Key decisions and trade-offs

- **Eager loading, not lazy.** `OrderService::paginate()` always calls `Order::query()->with('customer')` before applying filters, because every row in the list response needs the customer's name/phone/city – lazy loading would issue one query per row (classic N+1). The order-detail endpoint eager-loads `customer`, `statusHistories` and `statusHistories.user:id,name` in a single call.
- **Dashboard uses aggregate SQL only.** `DashboardService` never hydrates an `Order` model to compute its metrics – it issues one `GROUP BY status` count query and one `SUM(cod_amount)` query, so the endpoint's cost is independent of how many orders exist.
- **Row locking over optimistic checks.** I chose pessimistic locking (`lockForUpdate()`) for status transitions rather than an optimistic `updated_at` compare-and-swap, because transitions are infrequent per order and the simplicity of "read the true current state under a lock, then decide" outweighed the extra complexity of retry logic for what is, in practice, low contention on a single row.
- **Customer reuse by normalised phone.** Order creation matches an existing customer via `Customer::firstOrCreate(['phone' => $normalizedPhone])` and refreshes their name/city/address to the latest values supplied, on the assumption that the most recent order carries the customer's current details.
- **Sortable-column allowlist.** `OrderService::SORTABLE` restricts `sort_by` to `created_at` and `cod_amount` rather than passing the client's parameter straight into `orderBy()`, closing off arbitrary-column sort injection.

8 Frontend Implementation Steps

The frontend followed the same incremental, verify-as-you-go approach, again matching the git history:

1. **Scaffold config** – Vite + React 18 + Tailwind/PostCSS setup, the theme extension (colours, radius, shadows) declared once in `tailwind.config.js`, Inter loaded via Google Fonts, env variable `VITE_API_URL`.
2. **API/auth foundation** – the axios instance and interceptors, `AuthContext`, and small `lib/` helpers (`format.js` for MAD/date formatting, `errors.js` to normalise axios errors and Laravel 422 field errors, `orderStatus.js` as the single status-colour source of truth).
3. **Shared UI components** – `Button`, `Input`, `Select`, `Textarea`, `Card`, `Table` primitives, `StatusBadge`, `Alert`, `Pagination`, `ConfirmDialog`, `Toast`, `EmptyState`, `ErrorState`, `Spinner` – built before any page, so every page composes the same primitives rather than re-implementing borders/focus rings/spacing.
4. **App shell and routing** – `Layout` (fixed sidebar on desktop, off-canvas drawer on mobile), `Sidebar`, `ProtectedRoute`, and the router wiring in `App.jsx`.

5. **Pages** – Login (split screen), Dashboard (KPIs + status bars + recent orders), Orders List (toolbar + table + pagination), Create Order (two-section form), Order Details (summary + customer + change-status + timeline).
6. **Docs** – frontend README and the root run-the-stack instructions.

8.1 Notable implementation details

- **Debounced, cancellable search.** The orders list debounces the search input by 400ms before updating the API query, and every fetch creates a new `AbortController`, cancelling the previous in-flight request. This prevents a fast typist's earlier keystrokes from resolving *after* a later one and overwriting fresher results on screen. Any filter change resets to page 1; only page navigation itself is exempt.
- **Server-side everything.** Search, status, city, sort field and direction are all sent as query parameters on every change – there is no client-side filtering of an already-fetched page, keeping the frontend's behaviour correct regardless of dataset size.
- **Action buttons driven by the API, not hardcoded.** The order details page renders exactly one button per entry in the `allowed_next_statuses` array returned by the API – never a client-side copy of the state machine – styled orange for forward-progress transitions and red-outline for Cancel/Return, with a confirmation dialog before the PATCH is sent.
- **Field-level 422 mapping.** `CreateOrderPage` runs a client-side required-field check for instant feedback, then merges Laravel's `errors` object from a 422 response directly onto the matching input (e.g. the phone rule's rejection message), falling back to a form-level alert for anything that isn't a field error.
- **Loading / empty / error states on every data view.** Each page that fetches data renders one of: a skeleton or spinner while loading, a friendly icon-and-message empty state, or an error state with a retry button – including a distinct message for a genuine network failure (server unreachable) versus an API error response.

9 Extra Touches Beyond the Brief

After the core deliverable was complete and tested, a follow-up session added five small features, each chosen because it serves the real-world COD workflow the brief simulates. All are **strictly additive**: none changes an existing endpoint's response shape, the status state machine, or the phone validation rules – verified by the fact that all pre-existing tests continued to pass unmodified.

- **Customer lookup by phone** (`GET /api/customers/lookup`) – reuses the existing phone normaliser so a repeat customer can be found by typing their number, before an order is even submitted.
- **Existing-customer autofill on Create Order** – once a plausible phone number is typed (debounced) or the field loses focus, a banner offers to fill in the name/city/address from that customer's last order, saving re-entry for repeat COD buyers.
- **WhatsApp + call quick actions** on the order details page – COD confirmation in Morocco typically happens over a phone call or WhatsApp, so an ops agent can reach the customer in one click.
- **Copy-to-clipboard** for the order reference and phone number.
- **Relative dates with absolute-time tooltips** ("2 hours ago") in the orders table and status timeline, plus a dedicated **Cancelled-orders dashboard stat card**.

Each feature landed as its own commit with its own backend tests where applicable (the lookup endpoint added five feature tests), keeping the additive-only guarantee reviewable commit by commit.

10 Testing and Quality Assurance

The backend test suite runs under PHPUnit with `RefreshDatabase` against an in-memory SQLite database (configured in `phpunit.xml`), independent of whichever database the application uses at runtime. Beyond the automated suite, a dedicated QA and load-testing pass was run late in the project (chronology phase 8); its full results, including raw k6 output and `EXPLAIN QUERY PLAN` traces, live in the repository's `TESTING_REPORT.md` and are summarised here.

10.1 The automated test suite

Test class	Tests	What it asserts
<code>AuthTest</code>	3	401 on unauthenticated <code>GET /orders</code> ; a valid login returns a token; wrong credentials return 422.
<code>OrderCreationTest</code>	6	A valid order persists customer + order + one initial history row; phone sent as <code>0612345678</code> is stored as <code>+212612345678</code> ; a repeat phone reuses the existing customer; an invalid (landline) phone is rejected with no rows created; missing required fields are rejected; a negative <code>cod_amount</code> is rejected.
<code>OrderStatusTransitionTest</code>	4	An invalid transition (New→Delivered) returns 422 and writes no history row; a valid transition updates the order and writes exactly one history row; a terminal-status order rejects any transition; an unknown status value is a validation error.
<code>OrderListingTest</code>	6	Pagination meta is correct; the status filter limits results; an unknown status filter value returns 422; phone search matches both local and international format; sorting by <code>cod_amount</code> orders correctly; the show endpoint returns customer, history and <code>allowed_next_statuses</code> .
<code>DashboardTest</code>	3	All six status buckets are always present (even at zero); delivery rate is 0 when there are no delivered/returned orders (no division by zero); delivery rate and COD total are computed correctly for a known input.
<code>CustomerLookupTest</code>	5	The lookup endpoint (Section 9) finds a customer by phone in any accepted format, 404s cleanly on no match, and requires authentication.
<code>QueryCountAuditTest</code>	2	N+1 regression guard: the orders index issues a constant 3 queries and the show endpoint a constant 4, regardless of how many rows exist (measured live via <code>DB::listen()</code>).
<code>OrderStatusTest</code> (unit)	4	Every legal and illegal transition in the state machine; terminal statuses report no transitions.
<code>MoroccanPhoneTest</code> (unit)	16	Every accepted input shape (local, international, separators, double-zero prefix) normalises correctly; every rejected shape (landline 05, wrong length, foreign number, +212 followed by a trunk zero, non-numeric) returns null.

Latest run (`cd backend && php artisan test`):

```

1 Tests:   50 passed (145 assertions)
2 Duration: ~0.4s

```

This covers the three tests explicitly required by the brief (401 unauthenticated, valid order creation with phone normalisation, invalid transition returning 422 with no history row created)

plus the coverage listed above. Three of the 50 were added during the QA pass to close genuine gaps (the +2120... phone edge case, the negative-amount boundary, and the unknown listing filter); the rest predate it.

10.2 Quality-assurance tooling

The dedicated QA pass ran the following tools, each against the **local** environment only (never the deployed instance):

Tool	Result (one line)
Laravel Pint	10 files with cosmetic style issues, all fixed; <code>pint -test</code> now passes clean.
Larastan / PHPStan (level 6)	8 findings → 0: seven were tool false positives on enum casts (resolved with documented, zero-runtime-effect <code>@property</code> annotations), one was a genuine docblock/interface mismatch in the phone rule, corrected.
ESLint (react plugins)	5 findings: 2 fixed (stale directive, unescaped entity), 3 consciously deferred (<code>set-state-in-effect</code> on the standard fetch-on-mount pattern – an architectural rule, documented rather than restructuring tested pages).
npm audit	0 vulnerabilities.
Query audit	No N+1 anywhere; <code>EXPLAIN QUERY PLAN</code> confirmed the composite (<code>status</code> , <code>created_at</code>) and unique phone indexes are used exactly as designed.
k6 (3 load scenarios)	All thresholds passed: ~7,900 requests per read scenario at 75 req/s, p95 ≈ 17ms against a 500ms budget, 0% errors.

10.3 Load and concurrency testing (k6)

Three k6 scripts live in `backend/tests/load/`. Two exercise the read paths (orders list with randomised page/search/filter/sort parameters, and order details) under a ramp to 20 concurrent virtual users held for a minute – both passed their p95 < 500ms / error < 1% thresholds with two orders of magnitude to spare.

The third scenario is the most meaningful: **a real concurrency race on the status state machine**. Five virtual users simultaneously attempt the same `New→Confirmed` transition on one freshly created order. The verified outcome: exactly *one* request won (200), the other four were correctly rejected with 422 (by the time each acquired the row lock, the order was already `Confirmed`), the final history contained exactly two rows (creation + the single winning transition, no duplicates), and no request ever produced a 500. This is direct empirical evidence – not just a design argument – that the `lockForUpdate()` + re-validate pattern of Section 7 serialises concurrent writers correctly.

The scripts contain a hard safety guard: they throw immediately unless `BASE_URL` is `localhost/127.0.0.1`, so the free-tier deployed instance (Section 13) can never be load tested by accident.

10.4 Reproducing the full QA pass

```

1 cd backend
2 ./vendor/bin/pint --test # style
3 ./vendor/bin/phpstan analyse --memory-limit=512M # static analysis, level 6
4 php artisan test # 50 tests, 145 assertions
5
6 php artisan migrate:fresh --seed # fresh seeded DB, then:
7 PHP_CLI_SERVER_WORKERS=4 php artisan serve --port=8000
8 k6 run tests/load/login-and-list.js # (separate terminal)

```

```
9 k6 run tests/load/order-details.js
10 k6 run tests/load/status-change-contention.js
11
12 cd ../frontend
13 npm run lint && npx eslint src && npm audit && npm run build
```

11 AI Usage and Transparency

A dedicated `AI_USAGE.md` at the repository root is the authoritative, session-by-session transparency record; this section summarises it.

Tooling. The full implementation – backend and frontend – was built with **Claude Code**, an AI coding agent, operating directly in the project’s terminal and file system across several sessions (core build, CORS debugging, extra touches, Docker/deployment, QA & load testing). The five reference UI screens were themselves AI-generated (Stitch) and supplied as a starting visual reference, not written by hand.

Division of work. I (the author) directed the work throughout: supplying the functional requirements from the assessment brief, making the architectural calls the agent surfaced as decisions (Laravel 12 over an EOL Laravel 11; keeping the reference design’s layout but replacing its palette; React 18 and router v6 pinned deliberately against the toolchain’s newer defaults; switching the runtime database from SQLite to a local MAMP MySQL instance on request), and reviewing the generated code path by path rather than accepting it as a single opaque diff.

Verification performed. Every backend endpoint was exercised with live `curl` requests against a running server (login, order creation with a spaced/dashed phone number, an intentionally invalid landline phone, a valid and an invalid status transition, dashboard aggregates) before being considered done, in addition to the automated test suite. The frontend was driven end-to-end with a headless Chrome session (Puppeteer) exercising the real login flow, the orders list, order creation with a deliberately invalid phone number, and a live status transition with its confirmation dialog, confirming zero unexpected browser console errors. Database claims in this report were double-checked by querying the SQLite file and later the MySQL database directly with raw SQL, independent of the API layer. In the later sessions, the Docker image was genuinely built and run locally (a container runtime was installed specifically for that purpose – see Section 13) rather than the setup only being reviewed on paper, and every static-analysis finding in the QA pass was individually root-caused rather than blanket-suppressed.

What this means for review. Every specific claim in this report – file paths, method names, test names, migration indexes, package versions – is verifiable directly in the repository and was cross-checked against it while writing this document, rather than reconstructed from memory of the build process.

12 Challenges and Debugging

12.1 Laravel 11 was uninstallable

The brief specified Laravel 11, but `composer create-project laravel/laravel:^11.0` failed outright: every 11.x release falls inside the affected range of advisory `PKSA-mdq4-51ck-6kdq` (a CRLF injection in the default email validation rule), and Composer’s advisory policy blocks installation of any package version with an open, unfixed advisory by default. Checking Packagist’s advisory feed directly confirmed the fixed versions are only in the 12.60+ and 13.10+ ranges – there is no patched 11.x to fall back to, because the branch is end-of-life. Rather than silently adding the advisory ID to an ignore-list (which would ship a known vulnerability with no visible justification) or working around Composer entirely, I surfaced the trade-off explicitly and

proceeded on Laravel 12, which is a drop-in match for the brief's API surface. This is recorded in the backend README as a deliberate, justified deviation rather than an oversight.

12.2 Vite's default toolchain outpaced the requested stack

`npm create vite@latest - -template react` installed React 19 and react-router-dom v7 by default, but the brief was explicit about React 18 and router v6. Unlike the Laravel case there was no technical blocker forcing an upgrade, so I pinned back to `react@^18.3`, `react-dom@^18.3` and `react-router-dom@^6.30` immediately after scaffolding, before any application code was written against the newer APIs.

12.3 A monorepo restructure mid-build

The project began as a single Laravel repository at the root. Partway through the backend build, the requirement to also house a `frontend/` folder (and, later, the supplied `design/` assets) meant moving the entire Laravel tree into `backend/` and re-pointing git at the new root without losing history. This was done with `git mv` plus a re-staged `.gitignore` covering both projects, verified by confirming `vendor/`, `node_modules/` and both `.env` files were still excluded from version control after the move (a broad `git add -A` was checked with `git status` before committing, specifically to catch anything sensitive that the restructure might have accidentally re-included).

12.4 An empty git add pathspec silently dropped a commit's contents

While committing the frontend in logical, per-feature chunks, one `git add` call listed a config file path (`frontend/eslint.config.js`) that did not actually exist – Vite's scaffold had generated `.oxlintrc.json` instead. Git aborts the entire `add` when any listed pathspec doesn't match, so that commit silently contained none of the intended scaffold files, which only became visible when `git status` was re-checked before the next step. The fix was to re-stage the correct files explicitly in a follow-up commit; the lesson applied afterwards was to verify `git status` immediately after any multi-file `add`, not just before a final commit.

13 Deployment Journey and Infrastructure Decisions

13.1 Goal

Provide a **zero-cost live demo** so evaluators can try the application immediately, without any local setup – while keeping the fully documented local install (Section 15) as the primary, canonical way to run and review the project. Every hosting decision below was a conscious trade-off made against that goal and the delivery deadline; nothing here changed a line of application logic.

13.2 Option 1: Laravel Cloud (abandoned for a non-technical reason)

Laravel Cloud's Starter tier was the first choice, for obvious reasons: it is the framework vendor's own platform, with native Laravel support (no containerisation work at all) and free monthly usage credits that comfortably cover a demo of this size. The attempt ended at the billing step, before any engineering work was invested: the account's **payment-method verification failed with a Moroccan bank card**, a common situation for Moroccan-issued cards where international online payments are disabled or restricted by default by the issuing bank. This was a payment-verification blocker entirely external to the project – not a technical evaluation of the platform – so the pragmatic call was to move on immediately rather than spend deadline time on a banking workaround for a free tier.

TODO: Deployment specifics to confirm before submission: the exact date(s) of the deployment attempts, the precise wording/stage of the Laravel Cloud card-verification failure, and whether any other provider was considered between Laravel Cloud and Render.

13.3 Option 2: Render free tier via Docker (the final backend host)

Render’s free web-service tier requires no card at all and runs arbitrary Docker images, which dovetailed with the assessment’s own Docker bonus: the same `Dockerfile` serves as both the deployment vehicle and a deliverable in its own right.

The image is a two-stage build (`backend/Dockerfile`): a Composer stage that installs production dependencies, and a `php:8.4-apache` runtime stage serving Laravel’s `public/` docroot, with only `bcmath` added on top of the base image’s extensions. Two details are worth highlighting as deliberate:

- **Both stages run the same PHP version.** Composer bakes the PHP version it ran under into a permanent `vendor/composer/platform-check.php`; using the stock `composer:2` image’s own PHP for the build stage would produce an image that refuses to boot on the runtime PHP. The build stage therefore copies the Composer binary into a `php:8.4-cli` base instead.
- **Runtime port is substituted at container start, not build time.** Render assigns the listening port via `$PORT` only when the container starts, so the Apache config files ship with a `__PORT__` placeholder that `docker/docker-entrypoint.sh` rewrites before launching Apache.

The container was **verified locally before any deployment**: no container runtime existed in the working environment, so one was installed (Colima + the standalone Docker CLI) specifically so the image could be genuinely built and run rather than reviewed on paper. That verification surfaced three real bugs, each fixed and confirmed by re-running the container: the PHP-version platform-check mismatch above, `fakerphp/faker` missing from production dependencies (this app’s seeders run at every boot by design, see below), and a file-ownership bug where the SQLite database was created by `root` but written by Apache’s `www-data` worker – causing every write to fail with “attempt to write a readonly database” until ownership was corrected in the entrypoint.

13.4 Turning the free tier’s constraint into a feature

Render’s free tier has an **ephemeral filesystem**: everything written to disk – including the SQLite file – is wiped on every redeploy *and* every sleep/wake cycle. Rather than fight this (a paid persistent disk, or an external database, both out of scope for a zero-cost demo), the entrypoint embraces it: on every boot it recreates the SQLite file, fixes its ownership, and runs `php artisan migrate:fresh --seed --force`. For a *demo*, this is strictly better than persistence – every evaluator who wakes the instance gets the same clean, fully seeded 115-order dataset, and no amount of poking at the demo can leave it in a broken state for the next person. The same property would of course be disqualifying for production; the production path (managed MySQL/Postgres + a persistent volume) is documented in Section 14 territory rather than built, because the demo does not need it.

13.5 Frontend hosting and the final architecture

The React SPA is a static Vite build, so it needs no server at all: it is hosted on **Netlify**’s free tier (configured origin: `https://test-growmax.netlify.app`), with `VITE_API_URL` pointing at the Render backend. The backend’s CORS configuration (`config/cors.php`) reads the deployed

frontend origin from the `FRONTEND_URL` environment variable – set per environment in Render’s dashboard, never hardcoded – alongside the fixed local-dev allowlist.

Component	Final hosting
Frontend	Netlify free tier, static Vite build (no server-side rendering needed).
Backend API	Render free web service, the project’s own Docker image (Apache + PHP 8.4).
Database	SQLite inside the container, recreated and reseeded on every boot by design.
Auth	Sanctum bearer tokens (unchanged from local – no cookie/domain coupling to break across the two hosts).
Cost	\$0.

13.6 Trade-offs accepted, consciously

- **Cold starts.** Render free-tier services sleep after inactivity; the first request after a sleep can take on the order of a minute while the container boots (including the intentional reseed). Acceptable for a demo an evaluator opens once; unacceptable for production, and stated as such.
- **No data persistence.** By design, as above – a feature for a demo, a non-starter for production.
- **The deployed instance is never load tested.** All QA and k6 load testing (Section 10) ran exclusively against the local environment; the k6 scripts refuse non-localhost targets outright, so the free-tier instance cannot be hammered even by accident. Performance numbers in this report therefore describe the application’s behaviour, not Render’s free-tier hardware.
- **Bearer tokens over cookie-based SPA auth.** Chosen during the core build for simplicity, this also paid off at deployment time: token auth has no cookie-domain/SameSite coupling, so serving the frontend and backend from two unrelated domains (Netlify + Render) required nothing beyond the CORS origin entry.

14 Limitations and Possible Improvements

14.1 Honest gaps against the brief

- There is no password-reset, account-registration or multi-user-role flow – the brief only specified a single demo login, and none was built beyond it.
- The order-creation form’s “store name” is a free-text input rather than a managed list of stores; the API accepts any string.
- Rate limiting on `/login` is whatever Laravel’s default throttle middleware provides out of the box – it was not tuned or tested specifically for brute-force resistance.
- There is no soft-delete or order-cancellation-audit beyond the standard status-history trail (a cancelled order’s row is not distinguished from any other terminal order at the storage level beyond its `status` value).

14.2 Scaling to roughly 1,000,000 orders

The current design would degrade in specific, predictable ways at that volume, and the fixes are known rather than open research questions:

- **Pagination.** Laravel’s offset-based `LengthAwarePaginator` (`OFFSET ...LIMIT ...`) gets linearly slower as the offset grows, because the database must still scan and discard every

skipped row. At high page numbers on a million-row table this becomes the dominant cost. The fix is **cursor (keyset) pagination** – ordering and paging by (`created_at`, `id`) with a `WHERE (created_at, id) < (?, ?)` predicate instead of an offset – which Laravel supports via `cursorPaginate()`. This would require a small API contract change (an opaque cursor instead of a page number), so it was not adopted for a fifteen-per-page assessment dataset but is the first thing I would change.

- **Indexes already anticipate this.** The composite (`status`, `created_at`) index and the individual `cod_amount/created_at` indexes were added specifically so the listing endpoint's common filter+sort combinations stay index-backed rather than falling to a full table scan as the table grows; they do not need to change, only the pagination strategy sitting on top of them.
- **Search.** The current `LIKE '%term%'` search on `reference` and the joined customer's `name/phone` cannot use a standard B-tree index once the wildcard is leading. At scale this should move to a dedicated search index (MySQL/Postgres full-text search, or an external engine such as Meilisearch/Elasticsearch) rather than relational `LIKE`.
- **Dashboard aggregates.** The `GROUP BY status` count and `SUM(cod_amount)` queries are already index-backed and $O(\text{index size})$, not $O(\text{table size})$ in the naive sense, but at very high write volume I would move to a periodically refreshed summary table (or a queued job that maintains running counters) rather than computing the aggregate live on every dashboard request.
- **Background work.** Nothing in the current system needs a queue – every operation is synchronous and fast. At scale, notifying a courier integration or sending a customer SMS on each status change would move to Laravel's queue system so the HTTP response for a status change is not blocked on a third-party API call.

14.3 Concurrent status updates

The current implementation already addresses the most likely race directly: `OrderStatusService::transition()` re-reads the order **inside** the database transaction with `lockForUpdate()` before checking the state machine, so if two requests attempt to transition the same order at nearly the same instant, the second one blocks until the first commits, then re-validates against the *new* current status – it cannot succeed in applying a transition that is no longer legal, and cannot silently overwrite the first request's change. This is **pessimistic locking**, chosen because status transitions on a single order are infrequent and short-lived, making lock contention a non-issue in practice. This is no longer only a design argument: the QA pass's k6 contention scenario (Section 10) raced five simultaneous requests against the same order and confirmed exactly one winner, no duplicate history rows, and no 500s.

At larger scale, or if transitions became frequent enough for row-lock contention to matter, the alternative is **optimistic concurrency**: add a `version` integer (or reuse `updated_at`) to `orders`, have the update include `WHERE id = ? AND version = ?`, and treat a zero-row update as a conflict to retry or surface to the client. This trades a small amount of application-level retry logic for removing row locks from the write path entirely – worth doing only once lock waits are observed to be a real bottleneck, which they are not at this project's scale.

15 Installation Summary and Demo Credentials

Full steps are in the repository's `README.md` (root), `backend/README.md` and `frontend/README.md`. Summary:

```
1 # Backend (Terminal 1)
2 cd backend
3 composer install
```

```

4 cp .env.example .env
5 php artisan key:generate
6 touch database/database.sqlite
7 php artisan migrate --seed
8 php artisan serve                # http://localhost:8000
9
10 # Frontend (Terminal 2)
11 cd frontend
12 npm install
13 cp .env.example .env            # VITE_API_URL=http://localhost:8000/api
14 npm run dev                     # http://localhost:5173

```

Field	Value
Email	demo@growmax.ma
Password	password

16 Conclusion

The delivered system satisfies every core requirement of the brief: a Sanctum-authenticated Laravel API with a transaction-safe order status state machine expressed as a pure enum, Moroccan phone validation and normalisation enforced at both the request and model layers, a server-side filtered/sorted/ paginated order listing with eager loading to avoid N+1 queries, aggregate-only dashboard metrics, and a React frontend that drives every status action directly from the API's own `allowed_next_statuses` rather than duplicating the rules client-side. Fifty automated tests (145 assertions) cover the three explicitly required scenarios plus meaningful bonus coverage, and the whole stack was verified running end-to-end – not just built – via live HTTP requests and a headless-browser walkthrough of every page.

Beyond the core, the project was then hardened and shipped: five strictly additive workflow features, a dedicated quality-assurance pass (static analysis at Larastan level 6 brought to zero findings, an N+1/index audit confirming constant query counts, and three k6 load scenarios – including an empirical concurrency race proving the state machine's locking correct under simultaneous writers), and a zero-cost live deployment (Dockerised backend on Render, static frontend on Netlify). The deployment path was not a straight line – the first-choice platform was abandoned at the billing step for a payment-verification reason external to the project – and Section 13 documents that journey as it happened, because the choices made under a deadline (move on fast, turn the free tier's ephemeral filesystem into a self-resetting demo, never load test shared free hardware) are themselves part of the engineering record.

The deliberate deviations from the letter of the brief (Laravel 12 instead of an unpatched, EOL Laravel 11; React 18/router v6 pinned against a newer toolchain default) were each made for a stated, verifiable reason and documented at the point of decision rather than silently. The limitations section above is similarly meant as an honest account of what a longer engagement would add, not a list of oversights discovered after the fact.

Placeholders requiring your input before submission:

- Deployment specifics: attempt dates, the exact Laravel Cloud card-verification failure, any other providers considered, and the live backend URL (Section 13).